Exemplo 3: chat em tempo real em Rails e em Vetra

Um chat com salas, membros e mensagens em tempo real, implementado em Rails 8.1 (código real, com Hotwire e Action Cable) e em Vetra (sintaxe proposta, que ainda não existe). É o exemplo que exercita o pilar 4 — consultas reativas — e o caso mais difícil dele: revogar o acesso de quem está com a tela aberta.

Escopo

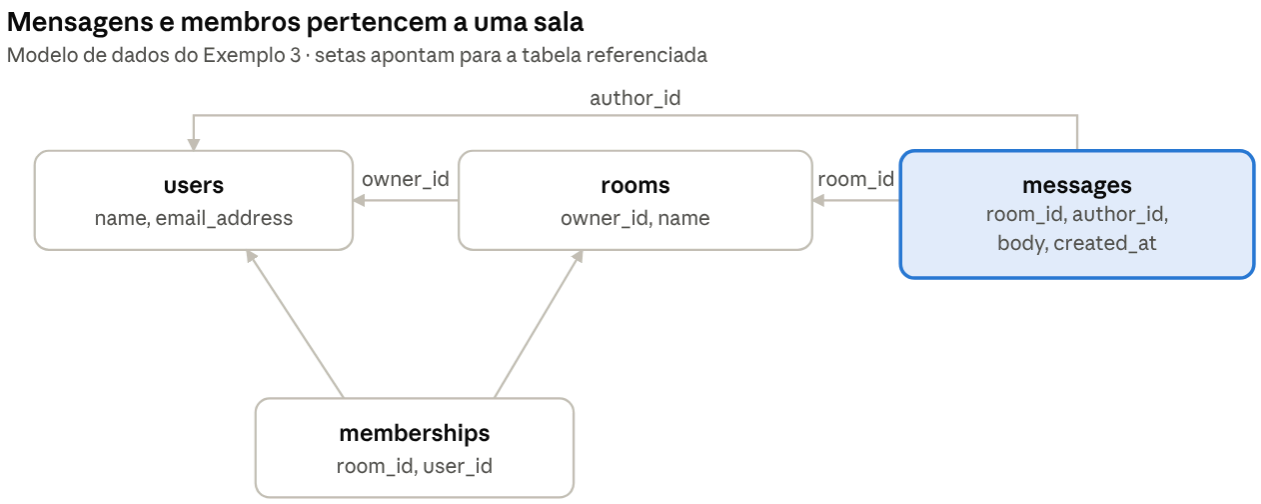
Um chat renderizado no servidor, com atualização em tempo real nas telas abertas.

- **Salas:** qualquer usuário logado cria uma sala e vira dono e membro dela; só membros veem e enviam mensagens.
- **Mensagens em tempo real:** uma mensagem nova aparece em todas as telas abertas da sala; excluir a própria mensagem a remove de todas as telas.
- **Membros:** o dono adiciona membros pelo e-mail e os remove.
- **Revogação:** quem é removido da sala deixa de receber novas mensagens, mesmo com a tela aberta.
- **Botão de excluir:** aparece só nas mensagens do próprio usuário.

Fora do exemplo nos dois lados: layout e CSS, indicador de "digitando", presença online, mensagens não lidas, anexos e paginação do histórico (a tela mostra as últimas 100 mensagens).

Arquitetura do projeto

O modelo de dados é o mesmo nos dois frameworks; no Rails são 17 arquivos escritos ou editados, incluindo um canal do Action Cable e um controller Stimulus, e no Vetra são 8.



modelo de dados do chat · 4 tabelas

A tabela de membros é escrita à mão no Rails (`memberships`) e gerada pelo `many_to_many :members` no Vetra (`room_memberships`).

As árvores abaixo mostram o projeto completo, como no explorador do editor. Marcações: `# escrito` é código deste exemplo; `# editado` é arquivo gerado e alterado à mão; itens sem marcação são gerados e não alterados. Pastas sem relação com o exemplo aparecem fechadas, com uma nota do que contêm.

Estrutura em Rails 8.1

```
chat/
├── .github/                                  # CI e Dependabot
├── .kamal/                                   # hooks e segredos do deploy
├── app/
│   ├── assets/                               # imagens e CSS
│   ├── channels/
│   │   ├── application_cable/
│   │   │   ├── channel.rb
│   │   │   └── connection.rb                 # gerado pela autenticação
│   │   └── room_channel.rb                   # escrito
│   ├── controllers/
│   │   ├── concerns/
│   │   │   └── authentication.rb             # gerado pela autenticação
│   │   ├── application_controller.rb
│   │   ├── memberships_controller.rb         # escrito
│   │   ├── messages_controller.rb            # escrito
│   │   ├── passwords_controller.rb           # gerado pela autenticação
│   │   ├── rooms_controller.rb               # escrito
│   │   └── sessions_controller.rb            # gerado pela autenticação
│   ├── helpers/
```

```
|   ├── javascript/
|   |   ├── controllers/
|   |   |   ├── application.js
|   |   |   ├── index.js
|   |   |   └── own_message_controller.js  # escrito
|   |   └── application.js
|   ├── jobs/
|   ├── mailers/                           # inclui passwords_mailer.rb
gerado
|   ├── models/
|   |   ├── concerns/
|   |   ├── application_record.rb
|   |   ├── current.rb                     # gerado pela autenticação
|   |   ├── membership.rb                  # escrito
|   |   ├── message.rb                     # escrito
|   |   ├── room.rb                        # escrito
|   |   ├── session.rb                     # gerado pela autenticação
|   |   └── user.rb                        # editado
|   └── views/
|       ├── layouts/
|       |   └── application.html.erb       # editado
|       ├── messages/
|       |   ├── _form.html.erb             # escrito
|       |   └── _message.html.erb          # escrito
|       ├── passwords/                     # gerado pela autenticação
|       ├── passwords_mailer/              # gerado pela autenticação
|       ├── pwa/
|       ├── rooms/
|       |   ├── index.html.erb             # escrito
|       |   └── show.html.erb              # escrito
|       └── sessions/                      # gerado pela autenticação
├── bin/                                   # rails, rake, setup, dev, jobs,
kamal, ci
├── config/
|   ├── environments/
|   ├── initializers/
|   ├── locales/
|   ├── application.rb                     # editado: idioma pt-BR
|   ├── boot.rb
|   ├── cable.yml                          # Solid Cable
|   ├── cache.yml
|   ├── ci.rb
|   ├── credentials.yml.enc
|   ├── database.yml
|   ├── deploy.yml
|   ├── environment.rb
```

```
|   ├── importmap.rb
|   ├── master.key
|   ├── puma.rb
|   ├── queue.yml
|   ├── recurring.yml
|   ├── routes.rb                          # editado
|   └── storage.yml
├── db/
|   ├── migrate/
|   |   ├── ..._create_users.rb            # gerado pela autenticação
|   |   ├── ..._create_sessions.rb         # gerado pela autenticação
|   |   └── 20261029120000_create_chat.rb  # escrito
|   ├── cable_schema.rb                    # tabela de mensagens do Solid
Cable
|   ├── cache_schema.rb
|   ├── queue_schema.rb
|   ├── schema.rb
|   └── seeds.rb
├── lib/
|   └── tasks/
├── log/
├── public/                                # páginas de erro, ícones,
robots.txt
├── script/
├── storage/
├── test/                                  # controllers, models, channels,
system
├── tmp/
├── vendor/
├── .dockerignore
├── .gitattributes
├── .gitignore
├── .rubocop.yml
├── .ruby-version
├── config.ru
├── Dockerfile
├── Gemfile                                # alterado pelo bundle add
├── Gemfile.lock
├── Rakefile
└── README.md
```

Estrutura em Vetra

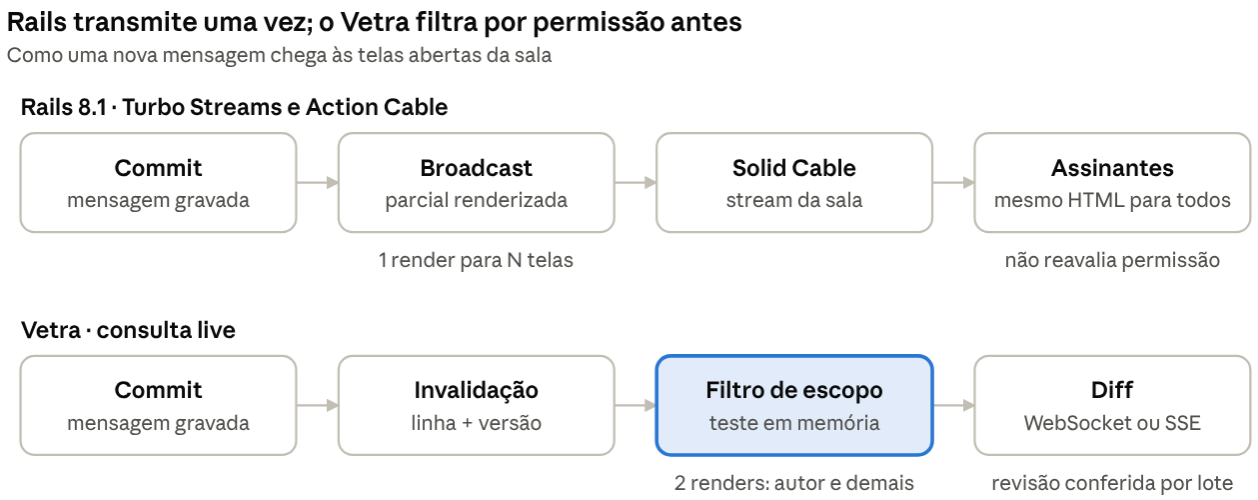
```
chat/
├── .vetra/
│   ├── cache/
│   └── graph.json                          # IR versionada do grafo
├── app/
│   ├── domains/
│   │   └── chat/
│   │       ├── add_member_by_email.rb      # escrito · command
│   │       ├── message.rb                  # escrito · entity
│   │       ├── queries.rb                  # escrito · query (live)
│   │       ├── roles.rb                    # escrito · roles
│   │       └── room.rb                     # escrito · entity
│   └── ui/
│       ├── assets/                         # CSS e imagens
│       ├── layouts/
│       │   └── application.html.erb
│       ├── templates/
│       │   └── rooms/
│       │       ├── index.html.erb          # escrito
│       │       └── show.html.erb           # escrito
│       └── pages.rb                        # escrito
├── bin/
│   ├── setup
│   └── vetra
├── config/
│   ├── environments/
│   │   ├── development.rb
│   │   ├── production.rb
│   │   └── test.rb
│   ├── locales/
│   │   └── pt-BR.yml
│   ├── app.rb                              # idioma, auth, ui, realtime
│   ├── database.rb                         # papéis do banco: aplicação e
migração
│   ├── secrets.enc                         # segredos cifrados
│   └── server.rb                           # Falcon (fibers: muitas conexões
abertas)
├── db/
│   ├── plans/                              # planos de migração expand/contract
│   ├── schema.sql                          # retrato do banco
│   └── seeds.rb
├── lib/
├── log/
├── public/
```

```
├── storage/
├── test/
│   ├── domains/
│   │   └── chat/                          # testes escritos pela equipe
│   ├── generated/                         # autorização e revogação gerados do
grafo
│   └── test_helper.rb
├── tmp/
├── .gitignore
├── .ruby-version                          # 4.0.9
├── config.ru
├── Dockerfile
├── Gemfile
├── Gemfile.lock
└── README.md
```

Usuários, sessões, login e o transporte de tempo real não aparecem na árvore do Vetra porque são gerados pelas opções `--auth=password` e `--realtime`.

Fluxo de tempo real

Os dois frameworks entregam a mensagem nova a todas as telas abertas, mas por caminhos opostos: o Rails empurra o mesmo HTML a todos, e o Vetra filtra cada mudança pelo escopo de cada tela antes de entregar.



fluxo de tempo real · Rails x Vetra

Com a entrega incremental (ADR-020), o Vetra não reconsulta o banco por tela: testa a linha nova contra o escopo de cada assinante em memória, confere a revisão de autorização uma vez por lote e renderiza uma vez por combinação de predicados do template. O custo fica próximo ao do broadcast; a diferença é que a permissão é conferida antes de cada lote.

Rails 8.1 — criação do projeto, banco de dados e tabelas

O Rails 8 já vem com Hotwire (Turbo e Stimulus) e Action Cable; em produção o transporte é o Solid Cable, que guarda as mensagens de broadcast numa tabela do PostgreSQL, e em desenvolvimento o `config/cable.yml` usa o adaptador `async`, em memória.

```
rails new chat --database=postgresql
cd chat
bundle add rails-i18n
bin/rails generate authentication
bin/rails generate migration CreateChat
bin/rails db:create db:migrate
```

```
# config/application.rb (linha adicionada dentro da classe Application)
config.i18n.default_locale = :"pt-BR"
```

```
# db/migrate/20261029120000_create_chat.rb
class CreateChat < ActiveRecord::Migration[8.1]
  def change
    add_column :users, :name, :string, null: false, default: ""

    create_table :rooms do |t|
      t.references :owner, null: false, foreign_key: { to_table: :users }
      t.string :name, null: false
      t.timestamps
    end

    create_table :memberships do |t|
      t.references :room, null: false, foreign_key: true
      t.references :user, null: false, foreign_key: true
      t.timestamps
      t.index %i[room_id user_id], unique: true
    end

    create_table :messages do |t|
      t.references :room, null: false, foreign_key: true
      t.references :author, null: false, foreign_key: { to_table: :users }
      t.text :body, null: false
      t.timestamps
      t.index %i[room_id created_at]
    end
  end
end
```

Rails 8.1 — models

Os broadcasts ficam nos callbacks do model, no estilo Hotwire; a revogação exige derrubar as conexões WebSocket do usuário removido.

```
# app/models/user.rb (gerado; linhas marcadas foram adicionadas)
class User < ApplicationRecord
  has_secure_password
  has_many :sessions, dependent: :destroy
  has_many :memberships, dependent: :destroy                    # adicionada
  has_many :rooms, through: :memberships                        # adicionada
  has_many :messages, foreign_key: :author_id, inverse_of: :author #
adicionada

  normalizes :email_address, with: ->(e) { e.strip.downcase }
end
```

```
# app/models/room.rb
class Room < ApplicationRecord
  belongs_to :owner, class_name: "User"
  has_many :memberships, dependent: :destroy
  has_many :members, through: :memberships, source: :user
  has_many :messages, dependent: :destroy

  validates :name, presence: true, length: { maximum: 80 }

  after_create { memberships.create!(user: owner) }

  def member?(user) = memberships.exists?(user: user)

  def owned_by?(user) = owner_id == user.id
end
```

```
# app/models/membership.rb
class Membership < ApplicationRecord
  belongs_to :room
  belongs_to :user

  validates :user_id, uniqueness: { scope: :room_id }

  after_destroy_commit :disconnect_user

  private

  # Derruba todas as conexões do usuário; ao reconectar, o RoomChannel
  # recusa a assinatura da sala da qual ele foi removido.
  def disconnect_user
    ActionCable.server.remote_connections.where(current_user:
user).disconnect
  end
end
```

```
# app/models/message.rb
class Message < ApplicationRecord
  belongs_to :room
  belongs_to :author, class_name: "User"

  validates :body, presence: true, length: { maximum: 4_000 }

  after_create_commit -> { broadcast_append_to room, target: "messages" }
  after_destroy_commit -> { broadcast_remove_to room }
end
```

Rails 8.1 — controllers, canal e rotas

O ponto crítico é o canal: o `turbo_stream_from` padrão aceita qualquer usuário logado que tenha o nome assinado do stream, e esse nome não expira. Por isso o exemplo usa um canal próprio que confere a participação na sala a cada assinatura.

```
# app/channels/application_cable/connection.rb (gerado pela autenticação)
module ApplicationCable
  class Connection < ActionCable::Connection::Base
    identified_by :current_user

    def connect
      set_current_user || reject_unauthorized_connection
    end

    private

    def set_current_user
      if (session = Session.find_by(id: cookies.signed[:session_id]))
        self.current_user = session.user
      end
    end
  end
end
```

```
# app/channels/room_channel.rb
class RoomChannel < ApplicationCable::Channel
  extend Turbo::Streams::Broadcasts, Turbo::Streams::StreamName
  include Turbo::Streams::StreamName::ClassMethods

  def subscribed
    stream_name = verified_stream_name_from_params
    room = GlobalID::Locator.locate(stream_name) if stream_name

    if room.is_a?(Room) && room.member?(current_user)
      stream_from stream_name
    else
      reject
    end
  end
end
```

```
# app/controllers/rooms_controller.rb
class RoomsController < ApplicationController
  def index
    @rooms = Current.user.rooms.order(:name)
    @room = Room.new
  end

  def show
    @room = Current.user.rooms.find(params[:id])
```

```
    @messages = @room.messages.includes(:author).order(:created_at).last(100)
    @message = Message.new
  end

  def create
    @room = Room.new(params.expect(room: [:name]).merge(owner: Current.user))
    if @room.save
      redirect_to @room
    else
      @rooms = Current.user.rooms.order(:name)
      render :index, status: :unprocessable_content
    end
  end
end
```

```
# app/controllers/messages_controller.rb
class MessagesController < ApplicationController
  def create
    room = Current.user.rooms.find(params[:room_id])
    message = room.messages.new(params.expect(message: [:body]).merge(author:
Current.user))

    if message.save
      render_form(room, Message.new)
    else
      render_form(room, message, status: :unprocessable_content)
    end
  end

  def destroy
    Current.user.messages.find(params[:id]).destroy!
    head :no_content
  end

  private

  # A mensagem nova chega pelo broadcast; a resposta só troca o formulário.
  def render_form(room, message, status: :ok)
    render turbo_stream: turbo_stream.replace(
      "new_message", partial: "messages/form", locals: { room: room, message:
message }
    ), status: status
  end
end
```

```
# app/controllers/memberships_controller.rb
class MembershipsController < ApplicationController
  before_action :set_owned_room

  def create
    user = User.find_by(email_address: params[:email_address])
    if user
      @room.memberships.find_or_create_by!(user: user)
      redirect_to @room, notice: "Membro adicionado."
    else
      redirect_to @room, alert: "Usuário não encontrado."
    end
  end

  def destroy
    @room.memberships.where.not(user_id: @room.owner_id)
         .find_by!(user_id: params[:id]).destroy!
    redirect_to @room, notice: "Membro removido."
  end

  private

  def set_owned_room
    @room = Current.user.rooms.find(params[:room_id])
    redirect_to @room, alert: "Só o dono gerencia membros." unless
@room.owned_by?(Current.user)
  end
end
```

```
# config/routes.rb (as duas primeiras rotas vêm do gerador)
Rails.application.routes.draw do
  resource :session
  resources :passwords, param: :token

  root "rooms#index"
  resources :rooms, only: %i[index show create] do
    resources :messages, only: :create
    resources :memberships, only: %i[create destroy]
  end
  resources :messages, only: :destroy
end
```

Rails 8.1 — views e JavaScript

O broadcast renderiza a parcial uma única vez, sem saber quem vai recebê-la; por isso o botão "Excluir" vai para todos, e um controller Stimulus o remove no navegador de quem não é o autor.

```
<%# app/views/layouts/application.html.erb (linha adicionada no <head>) %>
<meta name="current-user-id" content="<%= Current.user&.id %>">
```

```
<%# app/views/rooms/index.html.erb %>
<h1>Salas</h1>

<% @rooms.each do |room| %>
  <p><%= link_to room.name, room %></p>
<% end %>

<%= form_with model: @room do |f| %>
  <% @room.errors.full_messages.each do |message| %>
    <p><%= message %></p>
  <% end %>
  <%= f.text_field :name, placeholder: "Nome da sala" %>
  <%= f.submit "Criar sala" %>
<% end %>
```

```
<%# app/views/rooms/show.html.erb %>
<h1><%= @room.name %></h1>

<%= turbo_stream_from @room, channel: RoomChannel %>

<div id="messages">
  <%= render partial: "messages/message", collection: @messages %>
</div>

<%= render "messages/form", room: @room, message: @message %>

<% if @room.owned_by?(Current.user) %>
  <section>
    <h2>Membros</h2>
    <% @room.members.each do |member| %>
      <p>
        <%= member.name %>
        <% unless member == @room.owner %>
          <%= button_to "Remover", room_membership_path(@room, member),
method: :delete %>
        <% end %>
```

```
      </p>
    <% end %>
    <%= form_with url: room_memberships_path(@room) do |f| %>
      <%= f.email_field :email_address, placeholder: "E-mail do usuário" %>
      <%= f.submit "Adicionar" %>
    <% end %>
  </section>
<% end %>
```

```
<%# app/views/messages/_message.html.erb %>
<div id="<%= dom_id(message) %>" data-controller="own-message"
     data-own-message-author-id-value="<%= message.author_id %>">
  <strong><%= message.author.name %></strong>
  <time><%= l(message.created_at, format: :short) %></time>
  <p><%= message.body %></p>
  <%= button_to "Excluir", message_path(message), method: :delete, data: {
own: true } %>
</div>
```

```
<%# app/views/messages/_form.html.erb %>
<%= form_with model: [room, message], id: "new_message" do |f| %>
  <% message.errors.full_messages.each do |error| %>
    <p><%= error %></p>
  <% end %>
  <%= f.text_area :body, rows: 2 %>
  <%= f.submit "Enviar" %>
<% end %>
```

```
// app/javascript/controllers/own_message_controller.js
import { Controller } from "@hotwired/stimulus"

// Remove o botão "Excluir" das mensagens de outros autores.
export default class extends Controller {
  static values = { authorId: Number }

  connect() {
    const me = document.querySelector('meta[name="current-user-
id"]')?.content
    if (String(this.authorIdValue) !== me) {
      this.element.querySelector("[data-own]")?.remove()
    }
  }
}
```

Esconder o botão é só apresentação: a exclusão de verdade é protegida no servidor, porque `MessagesController#destroy` só encontra mensagens do próprio usuário.

Limites deste lado:

- Ao remover um membro, `disconnect_user` derruba todas as conexões dele, em todas as abas e salas. O navegador reconecta, e o `RoomChannel` recusa só a sala removida.
- As mensagens que já estavam na tela continuam visíveis até ele recarregar a página.
- Sem o canal próprio, o `turbo_stream_from` padrão continuaria entregando mensagens a quem guardou o nome assinado do stream.

Vetra — criação do projeto, banco de dados e entidades

Sintaxe proposta: nada abaixo existe ainda. A opção `--realtime` ativa as consultas live (pilar 4); as tabelas são derivadas das entidades.

```
vetra new chat --locale=pt-BR --auth=password --ui=server --realtime
cd chat
vetra db:create
vetra db:plan
vetra db:migrate
vetra dev
```

Arquivo gerado pelo `vetra new`, não escrito à mão:

```
# config/app.rb
Vetra.app :chat do
  locale   "pt-BR"
  auth     :password
  ui       :server
  realtime :websocket, fallback: :sse
end
```

```
# app/domains/chat/room.rb
entity :room do
  belongs_to :owner, entity: :user, default: :current_actor
  string :name, required: true, max: 80
  many_to_many :members, entity: :user, include: :owner

  crud
end
```

```
# app/domains/chat/message.rb
entity :message do
  belongs_to :room
  belongs_to :author, entity: :user, default: :current_actor
  text :body, required: true, max: 4_000

  crud
  index :room_id, :created_at
end
```

O que essas declarações geram: tabelas `rooms`, `messages` e a junção `room_memberships`; o dono entra como membro na criação da sala (`include: :owner`); os comandos `create_room`, `create_message`, `delete_message`, `add_member` e `remove_member`.

Vetra — papéis, consultas live e comando

O motor de consultas aplica o escopo dos papéis automaticamente: `rooms.order(:name)` já devolve só as salas das quais o usuário é membro. Com `live true`, a consulta vira assinatura.

```
# app/domains/chat/roles.rb
roles do
  authenticated do
    can :create, on: :room
    can :read,   on: :room,    where: { members: :current_actor }
    can :read,   on: :message, where: { room: { members: :current_actor } }
    can :create, on: :message, fields: %i[room_id body],
                 where: { room: { members: :current_actor } }
    can :delete, on: :message, where: { author: :current_actor }
    can :add_member, :remove_member, on: :room, where: { owner:
:current_actor }
  end
end
```

```
# app/domains/chat/queries.rb
query :my_rooms do
  uses :rooms
  call { |_input, rooms| rooms.order(:name) }
end

query :room_page do
  input :room_id, :id
  uses :rooms, :messages
  live true

  call do |input, rooms, messages|
    {
      room: rooms.with(:members).find!(input.room_id),
      messages: messages.where(room_id: input.room_id)
                        .order(:created_at).last(100).with(:author)
    }
  end
end
```

```
# app/domains/chat/add_member_by_email.rb
command :add_member_by_email do
  input :room_id, :id
  input :email, :email
  authorize :add_member, :room, record: :room_id
  uses :rooms, :users

  call do |input, rooms, users|
    user = users.find_by(email: input.email) or fail!(:email, "usuário não
encontrado")
    rooms.add_member!(input.room_id, user)
  end
end
```

Implícito pelo framework:

- **Atualização:** cada nova mensagem ou exclusão gera um evento com o `xid8` da transação. O motor testa em memória quais assinaturas de `room_page` ela afeta e renderiza uma vez por combinação de predicados do template (ADR-020).
- **Revogação:** remover um membro altera a tabela de membros, e um trigger incrementa a revisão de autorização. No modo `strict`, padrão aqui por serem mensagens pessoais, o processo lê essa revisão antes de cada lote: nenhum lote iniciado depois da remoção entrega dados ao ex-membro, e só a assinatura daquela sala é encerrada.

- **Sem nome de stream:** não existe nome de stream que possa ser guardado e reutilizado. A assinatura é a própria consulta, autorizada ao assinar e conferida pela revisão a cada lote.

Vetra — páginas e templates

O helper `live` marca o trecho da página que acompanha a consulta. Ele é renderizado por grupo de assinantes com a mesma permissão; como o template usa `can?`, o motor avalia esse predicado em memória para cada mensagem e agrupa pelo resultado. Cada mensagem nova gera duas renderizações — uma para o autor, com o botão "Excluir", e uma para os demais —, e não uma por tela, sem JavaScript.

```
# app/ui/pages.rb
page "/",          query: :my_rooms,  template: "rooms/index"
page "/rooms/:id", query: :room_page, template: "rooms/show", input: {
room_id: :id }
```

```
<%# app/ui/templates/rooms/index.html.erb %>
<h1>Salas</h1>

<% result.each do |room| %>
  <p><%= link_to room.name, "/rooms/#{room.id}" %></p>
<% end %>

<%= command_form :create_room do |f| %>
  <%= f.errors %>
  <%= f.text_field :name, placeholder: "Nome da sala" %>
  <%= f.submit "Criar sala" %>
<% end %>
```

```
<%# app/ui/templates/rooms/show.html.erb %>
<% room = result[:room] %>
<h1><%= room.name %></h1>

<%= live :messages, on_revoke: "Você não tem mais acesso a esta sala." do
|messages| %>
  <% messages.each do |message| %>
    <div>
      <strong><%= message.author.name %></strong>
      <time><%= l(message.created_at, format: :short) %></time>
      <p><%= message.body %></p>
      <% if can?(:delete, message) %>
        <%= command_button "Excluir", :delete_message, values: { id:
message.id } %>
```

```
      <% end %>
    </div>
  <% end %>
<% end %>

<%= command_form :create_message, values: { room_id: room.id }, reset: true
do |f| %>
  <%= f.text_area :body, rows: 2 %>
  <%= f.submit "Enviar" %>
<% end %>

<% if can?(:remove_member, room) %>
  <section>
    <h2>Membros</h2>
    <% room.members.each do |member| %>
      <p>
        <%= member.name %>
        <% unless member == room.owner %>
          <%= command_button "Remover", :remove_member,
                             values: { room_id: room.id, user_id: member.id }
%>
        <% end %>
      </p>
    <% end %>
    <%= command_form :add_member_by_email, values: { room_id: room.id } do
|f| %>
      <%= f.errors %>
      <%= f.email_field :email, placeholder: "E-mail do usuário" %>
      <%= f.submit "Adicionar" %>
    <% end %>
  </section>
<% end %>
```

Quando a revogação acontece, o trecho `live` é substituído pela mensagem de `on_revoke`. As mensagens que já estavam na tela somem junto.

Comparação

O Rails pede cerca de 250 linhas em 17 arquivos, incluindo JavaScript; o Vetra, cerca de 115 linhas em 8 arquivos, sem JavaScript. A diferença maior não está nas linhas, e sim na revogação de acesso.

Parte	Rails 8.1 (linhas aprox.)	Vetra (linhas aprox.)
Banco de dados e tabelas	30 (migração)	0 (derivado das entidades)
Models / entidades	45	17
Controllers, canal e rotas / papéis, consultas, comando e páginas	100	46
Views e JavaScript / templates	70	52
Total	**~250 em 17 arquivos**	**~115 em 8 arquivos**

Comportamento	Rails 8.1	Vetra (proposta)
Mensagem nova em todas as telas	`broadcast_append_to` no model	Consulta `live`
Botão "Excluir" só para o autor	Stimulus + meta tag no layout	`can?` no template
Autorização ao assinar	Canal próprio; o padrão não confere	Ao assinar e conferida pela revisão antes de cada lote
Revogação de membro	Derruba todas as conexões do usuário; mensagens antigas ficam na tela	Encerra só a assinatura da sala e limpa o trecho `live`
Nome de stream reutilizável	Existe e não expira; mitigado pelo canal próprio	Não existe
Custo por mensagem enviada	1 renderização para N telas	1 teste em memória por tela, 1 leitura de revisão por lote e 2 renderizações (autor e demais)

Leitura honesta:

- **Onde o Vetra ganha:** revogação garantida por lote, autorização conferida antes de cada entrega e interface por usuário sem JavaScript. É exatamente o problema que o pilar 4 promete resolver, e o Rails exige trabalho manual e conhecimento fino do Turbo para chegar perto.

- **Custo:** um teste em memória por tela, uma leitura de revisão por lote e duas renderizações por mensagem. A versão inicial desta proposta reconsultava o banco por tela — 1.000 consultas por mensagem numa sala com 1.000 telas abertas. Manter os grupos ainda tem custo extra, que cresce quando as permissões são muito individuais; a matriz de carga do gate G4 precisa medir isso.
- **Garantia honesta:** um lote iniciado antes da remoção pode terminar, e mensagens já entregues não podem ser recolhidas: somem da tela, mas podem ter sido copiadas. O Rails tem o mesmo limite.
- **Maturidade:** Hotwire e Action Cable estão em produção há anos. O ADR-018 reduz esse risco ao usar o próprio Turbo como cliente no navegador, mas o lado servidor do trecho `live` continua novo.
- **Sinais efêmeros:** especificados no ADR-019 ("digitando" com expiração, presença por leases por dispositivo), com a mesma autorização das consultas live, mas fora deste exemplo. No Rails, saem com um canal próprio, sem a garantia de revogação.